FREEDOM BROKER · CONVERSATION QUALITY ANALYTICS (CQA)

Подробная техническая документация проекта

Архитектура системы автоматической оценки качества клиентских чатов и звонков (RU / KZ / EN), математическая модель перераспределения весов N/A, контур апелляций и адаптация под Microsoft Dynamics 365.

Стек бэкенда	Каналы и языки	Стоимость 1 оценки ИИ	Интеграция CRM
Python 3 Stdlib + SQLite3	**Чаты и Звонки (RU/KZ/EN)**	**$0.00042 (≈ 0.21 ₸)**	**Dynamics 365 Web Resource**

1. Общая архитектура и технологический стек

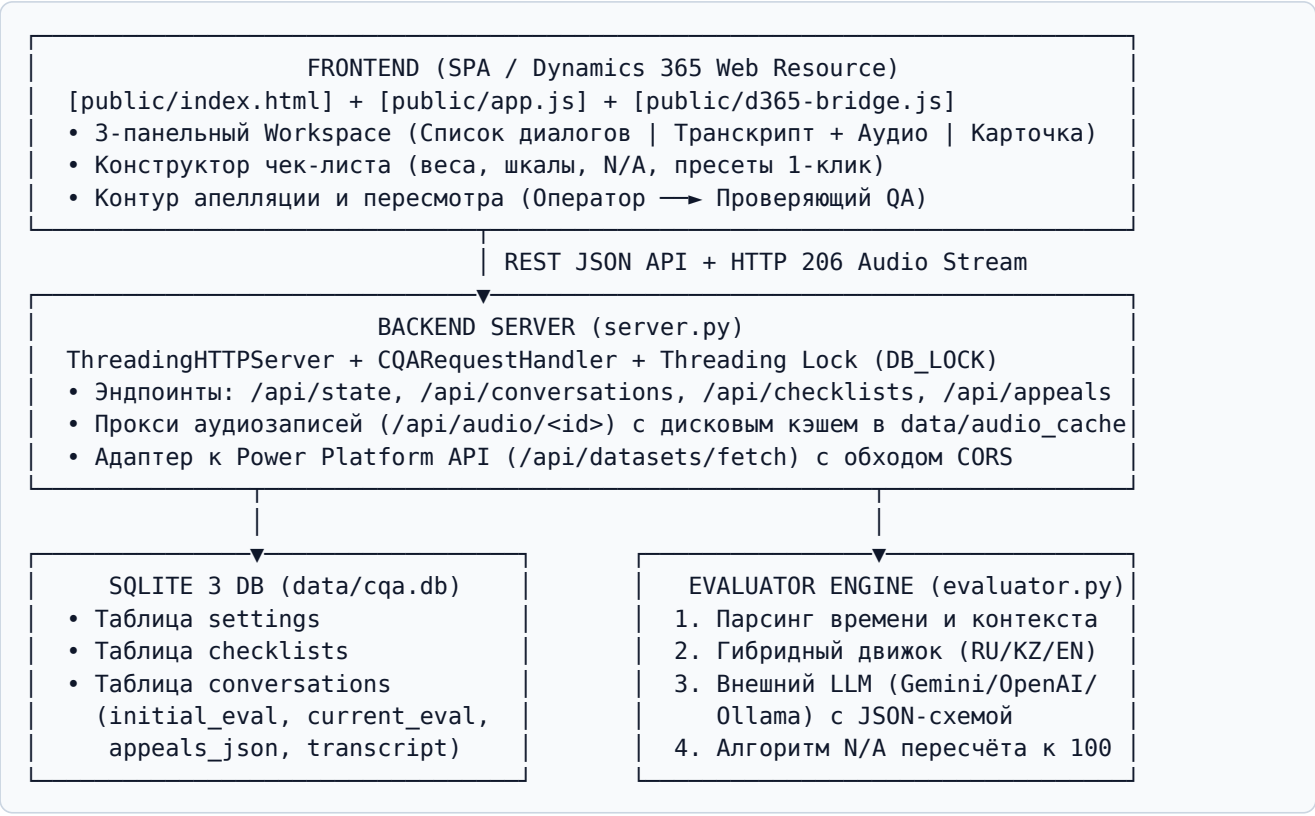
Проект разработан по принципу **Zero-Dependency Full-Stack Architecture** (без внешних Python- и npm-зависимостей). Серверная часть использует исключительно стандартную библиотеку Python 3 (`http.server` , `sqlite3` , `urllib.request` , `json` , `threading`), что гарантирует мгновенный запуск одной командой (`python3 server.py`), отсутствие конфликтов пакетов и полную готовность фронтенда к встраиванию как автономного **HTML Web Resource** в **Microsoft Dynamics 365 Customer Service Workspace**.

1.1. Структура файлов репозитория

Файл / Директория	Техническое назначение
`server.py`	Многопоточный HTTP/REST API сервер (`ThreadingHTTPServer`), менеджер транзакций SQLite 3, серверный адаптер к API организаторов (Power Automate) и потоковый аудио-прокси с поддержкой `HTTP 206 Partial Content` .
`evaluator.py`	Ядро аналитической и LLM-оценки диалогов (RU / KZ / EN), парсер относительных и ISO-8601 таймкодов, математический движок расчёта баллов и пропорционального перераспределения весов при `N/A` .
`public/index.html`	SPA-интерфейс приложения: 4 основных экрана (рабочее место оценки, конструктор чек-листа, центр апелляций, аналитика и стоимость ИИ) + панель 5-минутной защиты и модальные окна.
`public/app.js`	Клиентская логика (`window.App`), управление состоянием, синхронизация транскрипта с HTML5-аудиоплеером, конструктор критериев со шкалами и интерактивная трассировка пересмотра оценок.
`public/d365-bridge.js`	Адаптер интеграции с Microsoft Dynamics 365 (`window.D365Bridge`): чтение контекста `Xrm.Page` , разбор параметра `?data=` и отправка событий через `window.postMessage` .
`public/styles.css`	Дизайн-система в корпоративной палитре Freedom Broker и стили оболочки Dynamics 365 Customer Service Workspace.
`dev_dataset.json`	Локальный эталонный кэш набора `DEV` (65 диалогов: 35 RU, 15 KZ, 15 EN; 60 чатов и 5 телефонных звонков с аудиозаписями).

`data/cqa.db`	Реляционная БД SQLite 3 (автоматически инициализируется и заполняется при первом старте) и директория `data/audio_cache/` для кэширования WAV-потоков звонков.

1.2. Схема взаимодействия подсистем



2. Математическая модель оценки и перераспределения весов (N/A)

В функции `calculate_checklist_score(checklist, criteria_results)` (`evaluator.py`, строки 248–436) реализован точный 5-шаговый алгоритм расчёта баллов со страницы 2 спецификации чек-листа Freedom Broker.

2.1. Формулы расчёта по шагам

1. **Проверка допустимости `N/A`**: для каждого критерия i проверяется булев параметр `allow_na`. Если для критерия с `allow_na == False` (например, `01 Установление контакта`, `05 Эмпатия`, `06 Управление диалогом`) пришёл ответ `"N/A"`, система принудительно заменяет его на `"НЕТ"` (или `0` для шкального критерия), исключая невалидное обнуление знаменателя.

2. **Вычисление сырых баллов ($P_{\text{raw},i}$)**:
   - Для бинарного критерия (`type == "yes_no"`) с базовым весом W_i:

$$P_{\text{raw},i} = \begin{cases} W_i & \text{если ответ} \in \{\text{ДА}, \text{YES}, \text{ИӘ}\} \\ 0 & \text{если ответ НЕТ} \end{cases}$$

   - Для шкального критерия (`type == "scale"`) с выбранной ступенью $S_{\text{selected},i}$ и максимальной ступенью шкалы $S_{\text{max},i}$ (пропорциональный перевод ступени в вес критерия):

$$P_{\text{raw},i} = \frac{S_{\text{selected},i}}{S_{\text{max},i}} \times W_i$$

Пример: Критерий `06 Управление диалогом` имеет шкалу $0..10$ ($S_{\max} = 10$) и вес $W_6 = 15$. При выборе ступени 7 («Один заметный сбой, но клиент может понять ответ») сырой балл составляет $\frac{7}{10} \times 15 = 10.5$ балла.

- Для критического флага (`type == "critical_flag"`, критерий `08 Грубые ошибки`): базовый вес $W_8 = 0.0$. При ответе `"ДА"` поднимается флаг `has_critical_error = True` и формируется запись в массиве `critical_errors` без искажения 100-балльной шкалы остальных критериев.

3. **Исключение критериев с `N/A` и пропорциональное перераспределение весов к 100 баллам**: Пусть A — множество применимых весовых критериев диалога (где ответ $\neq \mathrm{N/A}$ и $W_i > 0$). Сумма применимых базовых весов равна:

$$W_{\text{app}} = \sum_{i \in A} W_i$$

Тогда **эффективный вес** каждого применимого критерия $W_{\text{eff},i}$ и его **перераспределённый балл** $P_{\text{redist},i}$ вычисляются пропорционально так, чтобы сумма эффективных весов всегда составляла ровно **100.00 баллов**:

$$W_{\text{eff},i} = \frac{W_i}{W_{\text{app}}} \times 100, \quad P_{\text{redist},i} = \frac{P_{\text{raw},i}}{W_{\text{app}}} \times 100$$

4. **Итоговый балл диалога (`final_score`)** округляется до двух знаков после запятой:

$$\text{Final Score} = \text{round}\left(\frac{\sum_{i \in A} P_{\text{raw},i}}{W_{\text{app}}} \times 100, 2 \right)$$

2.2. Практический пример перераспределения весов на диалоге `p1-ru-025`

В диалоге `p1-ru-025` клиент покидает чат сразу после приветствия оператора, не сформулировав вопрос.

- Критерии `02 Выявление потребности` (15 б.), `03 Идентификация` (20 б.), `04 Консультация` (20 б.) и `07 Завершение` (5 б.) получают законный статус `N/A` (сумма исключённых весов `na_weight_redistributed` = **60.0 баллов**).
- Оставшиеся применимые критерии: `01 Установление контакта` ($W_1 = 10$), `05 Эмпатия` ($W_5 = 15$), `06 Управление диалогом` ($W_6 = 15$). Сумма применимых весов $W_{\text{app}} = 40.0$ баллов.
- После пропорционального масштабирования ($\times \frac{100}{40} = \times 2.5$) эффективные веса становятся равными $W_{\text{eff},1} = 25.0$, $W_{\text{eff},5} = 37.5$, $W_{\text{eff},6} = 37.5$ (в сумме `100.00`), и оператор справедливо получает **100.00 / 100**.

3. Устройство движка оценки (`evaluator.py`)

Движок поддерживает два режима: **встроенный гибридный многоязычный движок `Freedom-CQA-Hybrid`** (работает локально за `< 3 мс` на диалог без внешних ключей) и **внешний LLM-режим** (`evaluate_conversation_llm`).

3.1. Нормализация таймкодов и обогащение транскрипта

В данных встречаются два формата времени: относительный формат чатов (`"MM:SS"` или `"HH:MM:SS"`) и абсолютный ISO-8601 формат телефонных звонков из Power Platform с 7-значными долями секунды (`"2026-02-02T11:25:14.1234567Z"`). Функции `parse_timestamp_seconds` и `enrich_transcript_with_offsets` (`evaluator.py`, строки 186–245):

- Усекают 7-значные микросекунды `.NET` до 6 знаков для совместимости с `datetime.fromisoformat`;
- Вычисляют для каждой реплики точное смещение `offset_seconds` от старта записи звонка (реплика 0);
- Добавляют поля `turn_index`, `offset_seconds` и `formatted_offset` (`MM:SS`), используемые фронтендом для перемотки аудиозаписи при клике на доказательство.

3.2. Извлечение структурных сигналов (`analyze_conversation_context`)

Перед проверкой критериев функция `analyze_conversation_context` (строки 583–647) разделяет реплики по ролям (`operator` , `client` , `bot` , `system`), сохраняя реплики бота и системы как обязательный контекст:

- Находит системную реплику эскалации (`"эскалация"` , `"жіберілді"` , `"transferred"` , `"присоединился"`) и вычисляет задержку первого ответа оператора `first_response_wait` (в секундах).
- Выделяет содержательные обращения клиента (`substantive_client_turns`), отфильтровывая нажатия цифр в голосовом меню IVR (`"1"` , `"2"`) и сервисные реплики переключения на оператора.

3.3. Алгоритмы оценки 8 базовых критериев на RU, KZ и EN (`_evaluate_standard_8`)

Код	Критерий	Тип / Вес	Техническая логика проверки и паттерны
`01`	Установление контакта	`yes_no` 10 б. (без N/A)	Проверяет первую реплику оператора (`first_op`) по регулярным выражениям `GREETING_PATTERNS` на языке клиента (`RU` / `KZ` / `EN`) и `INTRO_PATTERNS` (включая `[Имя скрыто]`). Отсекает фамильярные приветствия `INFORMAL_GREETING_ONLY` (**«Привет»**, **«Сәлем»**).
`02`	Выявление потребности	`yes_no` 15 б. (`allow_na`)	Ставит `N/A` , если данных хватало с самого начала или клиент покинул чат до вопроса. Ставит `НЕТ` , если оператор переспросил деталь, уже написанную клиенту боту, либо дал ответ вслепую без уточнения платформы/тикера.
`03`	Идентификация	`yes_no` 20 б. (`allow_na`)	Сопоставляет запросы по счёту (`ACCOUNT_SPECIFIC_PATTERNS`) с системным подтверждением личности (`ID_VERIFIED_PATTERNS`) или голосовой верификацией (ФИО + ИИН + SMS-код ОТП) строго **до** раскрытия информации по счёту.
`04`	Корректная и полная консультация	`scale` 0–20 Вес 20 б. (`allow_na`)	Оценивает полноту решения по ступеням `20 / 15 / 10 / 5 / 0` . Например, снижает до `10` , если клиент задал два существенных вопроса (раздел тарифа и размер комиссии), а оператор ответил только на один; `15` — если упущена второстепенная деталь (комиссия ОТС или уточняющий вопрос).
`05`	Эмпатия и клиентоориентированность	`yes_no` 15 б. (без N/A)	Проверяет вежливость тона и адресную реакцию на жалобу/неудобство клиента (например, фиксирует нарушение при грубости, отписках или механическом 4-кратном повторении одной фразы в звонке `p4-ru-004`).
`06`	Управление диалогом, скорость и грамотность	`scale` 0–10 Вес 15 б. (без N/A)	Оценивает ясность и структуру ведения диалога по ступеням `10 / 7 / 4 / 0` . Снижает до `4` , если из-за путаного объяснения клиенту пришлось переспрашивать маршрут; до `7` — при одном заметном сбое или длительном удержании на линии без возврата.
`07`	Завершение	`yes_no` 5 б. (`allow_na`)	`N/A` — при обрыве связи или выходе клиента (`CLIENT_ABORT_PATTERNS`). `ДА` — оператор подвёл итог / предложил дальнейшую помощь **и** попрощался. `НЕТ` — закрыл чат без прощания либо только попрощался без резюме и предложения помощи.

`08`	Грубые ошибки	`critical_flag` 0 б. (без N/A)	Сканирует все реплики оператора по 3 группам критических паттернов на RU/KZ/EN: 1) `INSULT_PATTERNS` (оскорбления), 2) `BYPASS_COMPANY_PATTERNS` (перевод на личную карту / платная частная консультация), 3) `UNVERIFIED_DISCLOSURE_PATTERNS` (раскрытие статуса счёта без верификации).

Каждая оценка формирует массив доказательств `evidence` (функция `_make_evidence`: `turn_index`, `timestamp`, `formatted_offset`, `offset_seconds`, `speaker`, `text`, `note`) и, в случае ошибки, поле `missing_action_location` с точным указанием реплики, где следовало выполнить действие.

3.4. Динамическая оценка новых и модифицированных критериев (`evaluate_custom_or_modified_criterion`)

При изменении чек-листа через UI (включая замену критерия на контрольной проверке жюри) функция `evaluate_custom_or_modified_criterion` (строки 1168–1383):

1. Автоматически адаптирует формат ответа при смене типа критерия (например, конвертирует `ДА/НЕТ` в числовую шкалу или масштабирует оценку к новым ступеням `scale`).
2. Распознаёт семантику новых пользовательских критериев по названию и описанию:
   - **Оперативность / SLA первого ответа**: извлекает норматив в секундах/минутах из описания критерия (`r"(\d+)\s*минут"`) и сравнивает его с реальным временем ожидания `first_response_wait` между репликой эскалации и первой репликой оператора;
   - **Обучение навигации и самообслуживанию**: проверяет наличие точных маршрутов в интерфейсе (`→`, `tradernet.global`, разделы меню и отчётов);
   - **Упрощённое завершение (только прощание)** и любые другие пользовательские критерии.

3.5. Интеграция с внешними LLM и расчёт стоимости 1 оценки (`evaluate_conversation_llm`)

Система поддерживает подключение внешних моделей (`Google Gemini 2.5 Flash`, `OpenAI GPT-4o-mini`, `OpenRouter`, локальная `Ollama`) через единый структурированный JSON-запрос:

- **Количество вызовов на 1 оценку диалога**: **1 вызов** (в системном промпте передаётся спецификация всех критериев активного чек-листа и пронумерованный транскрипт `[turn_index] timestamp (speaker): text`).
- **Средний объём контекста на 1 диалог**: `~1 150 входных токенов` + `~420 выходных токенов`.
- **Стоимость 1 оценки (на примере `gemini-2.5-flash`)**:

$$\text{Cost} = 1150 \times \$\frac{0.15}{10^6} + 420 \times \$\frac{0.60}{10^6} \approx \$0.000425 \text{ USD} \approx 0.21₸$$

  Оценка всего набора из 65 диалогов обходится примерно в **$0.027 USD (≈ 13.6 ₸)**.

4. Серверная часть, база данных SQLite и аудио-прокси (`server.py`)

4.1. Схема базы данных SQLite (`data/cqa.db`)

Доступ к БД синхронизирован через `DB_LOCK = threading.Lock()`. Схема включает 3 таблицы:

- `settings (key TEXT PRIMARY KEY, value_json TEXT)` — настройки провайдера ИИ, модели, API-ключа и тарифов токенов.
- `checklists (id TEXT PRIMARY KEY, name TEXT, description TEXT, is_active INTEGER, config_json TEXT, updated_at TEXT)` — хранит базовый и пользовательские чек-листы.
- `conversations`:
  - `conversation_id TEXT PRIMARY KEY`, `dataset_group_id TEXT`, `channel TEXT`, `language TEXT`, `audio_url TEXT`;
  - `transcript_json TEXT` — исходный JSON-массив реплик;

- `initial_eval_json TEXT` — **неизменяемый слепок первоначальной оценки ИИ** (гарантирует сохранность исходного результата до любых апелляций);
- `current_eval_json TEXT` — **актуальная оценка** (обновляется при пересмотре проверяющим QA);
- `appeals_json TEXT` — массив объектов апелляций и решений проверяющего.

4.2. Спецификация REST API эндпоинтов

Метод и URL	Функция в `server.py`	Описание работы
`GET /api/state`	`_handle_get_state`	Возвращает активный чек-лист, пресеты критериев, конфигурацию LLM и агрегированную статистику по языкам, каналам, `N/A` и апелляциям.
`GET /api/conversations`	`_handle_list_conversations`	Список диалогов с фильтрацией по `group_id`, `language`, `channel`, быстрым фильтрам (`critical`, `na`, `appeals`, `low_score`) и поиску по тексту.
`GET /api/conversations/<id>`	`_handle_get_conversation`	Полная карточка диалога: обогащённый транскрипт с `offset_seconds`, URL аудио-прокси, `initial_evaluation`, `current_evaluation` и `appeals`.
`POST /api/datasets/fetch`	`_handle_fetch_dataset`	Загружает набор диалогов из API организаторов (`ORGANIZER_API_BASE_URL`) по параметру `group_id` на стороне сервера (устраняя ошибки браузерного CORS) и сразу оценивает их.
`POST /api/conversations/upload`	`_handle_upload_conversations`	Принимает загруженный пользователем JSON-файл с диалогами, сохраняет в SQLite и запускает оценку по активному чек-листу.
`POST /api/checklists`	`_handle_save_checklist`	Сохраняет изменённый чек-лист как активный и пакетно переоценивает все выданные диалоги менее чем за `0.2 сек`.
`POST /api/evaluate`	`_handle_evaluate`	Принудительный пересчёт одного диалога (`conversation_id`) или всего набора (`all: true`).
`POST /api/appeals/submit`	`_handle_submit_appeal`	Сохраняет апелляцию оператора по спорному критерию со статусом `pending`.
`POST /api/appeals/decide`	`_handle_decide_appeal`	Фиксирует решение проверяющего (`confirm` или `change` с новым ответом), сохраняет `initial_evaluation` внутри критерия и автоматически пересчитывает итоговый балл через `calculate_checklist_score`.
`GET /api/audio/<id>`	`_handle_audio_proxy`	Потоковый прокси аудиозаписей телефонных звонков с локальным кэшированием (`data/audio_cache/<id>.wav`) и поддержкой `HTTP 206 Partial Content`.

4.3. Потоковый аудио-прокси с поддержкой `Range` (`HTTP 206 Partial Content`)

Для того чтобы HTML5-плеер `<audio>` в браузере мог мгновенно перематывать запись звонка на секунду доказательства (`audio.currentTime = offsetSeconds`), сервер обязан поддерживать побайтовые `Range`-запросы. В методе `_handle_audio_proxy` (`server.py`, строки 1048–1136):

1. При первом обращении к `/api/audio/<id>` сервер скачивает аудиозапись по `audio_url` организаторов и сохраняет её в `data/audio_cache/<id>.wav`.
2. При получении заголовка `Range: bytes=start-end` сервер вычисляет срез байтов `chunk_len = end - start + 1`, выполняет `f.seek(start)` и возвращает статус `206 Partial Content` с заголовками `Accept-Ranges: bytes` и `Content-Range: bytes start-end/file_size`.

5. Клиентская часть и интеграция с Microsoft Dynamics 365

5.1. Архитектура фронтенда (`public/app.js` и `public/index.html`)

Клиентское приложение построено вокруг модуля `window.App` и разделено на два взаимосвязанных контура:

- **Контур 1 (Настройка и автоматическая оценка)**:
  - Вкладка `Конструктор чек-листа` позволяет редактировать названия, секции, формулировки, веса, переключать тип ответа (`ДА / НЕТ`, `Числовая шкала 0–N` с редактированием каждой ступени, `Грубая ошибка`) и флаг `allow_na`, а также применять готовые пресеты замены критерия в 1 клик.
  - Кнопки `API (group_id)` и `Загрузить JSON` обеспечивают загрузку как базового набора `DEV`, так и контрольного `group_id` на 23-м часу хакатона.
- **Контур 2 (Работа с результатом, апелляция и пересмотр)**:
  - 3-панельный экран `Диалоги и Оценка` связывает карточку оценки с транскриптом и аудиоплеером: клик по любой плашке доказательства вызывает `App.jumpToTurn(turnIndex, offsetSeconds)`, плавно прокручивает транскрипт, подсвечивает реплику и перематывает аудиозапись звонка.
  - В карточке каждого оспоренного критерия и во вкладке `Апелляции и Пересмотр` отображается полная неизменяемая цепочка аудита из 4 блоков: **1. Исходная оценка ИИ** → **2. Обращение (Апелляция оператора)** → **3. Решение проверяющего QA** → **4. Пересчитанный финальный результат**.

5.2. Мост интеграции с Microsoft Dynamics 365 (`public/d365-bridge.js`)

Модуль `window.D365Bridge` обеспечивает нативную совместимость приложения при встраивании в карточку диалога **Dynamics 365 Customer Service Workspace** (сущность `msdyn_ocliveworkitem`):

1. Определяет наличие родительского окружения `window.Xrm` или `window.parent.Xrm` (либо активирует визуальную оболочку предпросмотра по параметру `?d365=1` / кнопке `D365 Режим`).
2. Декодирует стандартный параметр веб-ресурса Dynamics 365 `?data=conversation_id%3Dp1-ru-001` и автоматически открывает нужный диалог при открытии формы в CRM.
3. Отправляет стандартизированные сообщения `window.parent.postMessage` (`source: "FreedomBroker.CQA.WebResource"`, события `CONVERSATION_LOADED`, `APPEAL_SUBMITTED`, `APPEAL_DECIDED`) в родительскую форму Dynamics 365 при выборе диалога и пересмотре оценки.